

End to End Testing Plan of “xalts” Web Application

1. Introduction:

End to end testing ensure the complete application flow functions as expected, covering all components and integration. This test plan various features covered, testing types, their objectives, test cases and both success and failure scenarios.

2. Features Covered:

1. User Signup.
2. User Sign-in.
3. Submit Requests to Existing Blockchain.
4. Submit Requests to Create New Private Blockchain.

3. Testing Types and Objectives:

- **Functional Testing:**

Objectives: Validate that the application functions as expected based on requirements.

- **Test Case 1:** User can successfully sign-up with valid information.(Email, password)
Success: User is redirect to the dashboard.
Account is created and confirmation message displayed.
Failure: Account is not created, and an appropriate error message is shown.
- **Test Case 2:** User can successfully sign in with valid credentials.
Success: User is redirect to the dashboard.
Failure: User receives an error message for incorrect credentials and sign-up button should be disable .
- **Test Case 3:** User attempts to sign up with existing email.
Success: System prevents account creation and shows a duplicate error.
Failure: Account is created again.
- **Test Case 4:** User can sing out sucessfully.
Success: User is redirected to login page.
Failure: User remains logged in or sees error.
- **Test Case 5:** User attempts to sign in with a non-existing email.
Success: System denies access and show appropriate error message.
Failure: User is granted access.
- **Test Case 6:** User submit request to exisiting blockchain
Success: Nodes added successfully with correct node id and public IP.
Wallet added with appropriate permissions.
Submission succeeds and conformation is shown.
Failure: Invalid Node id and IP format reject with validation message and NEXT button will be disabled.
Empty or duplicate node fields are blocked.
Missing wallet address rejected with validation message.
Submission fails with validation errors or server side issues.
- **Test Case 7:** User submit requests to new private blockchain.
Success: User enters valid network name and wallet address.
Nodes and wallets are added correctly with valid format.
Review and submission succeed conformation is shown.
Failure: Missing or invalid network name or wallet format.
Invalid node id or public IP format is rejected.

Submission blocked due to error in submission form.

● Integration Testing

Objective: Ensure modules communicate correctly.

- **Test Case 1:** Data from the Database correctly reflects in the UI.
Success: Data is displayed correctly.
Failure: Data mismatch or missing records.
- **Test Case 2:** API request return expected response.
Success: API response matches expected format.
Failure: API returns incorrect or empty response.
- **Test Case 3:** Submit requests to existing blockchain.
Success: Transaction is successfully validated and conformed on the existing block chain.
Failure: API rejects transaction, or block chain node fails to respond.
- **Test Case 4:** Submit requests to create a new private blockchain.
Success: New block chain instance is created and initialized.
Failure: Creation failed due to permission error, invalid configuration, time out.

● UI/UX Testing

Objective: Ensure user interface functions correctly.

- **Test Case 1:** Responsive design to different screen size.
Success: UI adjust properly.
Failure: UI element break or overlap.
- **Test Case 2:** Button function correctly.
Success: Clicking a button triggers expected action.
Failure: Button does not respond or redirects incorrectly.
- **Test Case 3:** User can easily identify navigation. (Back to previous page or next page)
Success: Navigation icons are visible and responsive.
Failure: Invisibility of navigation icons.

● Performance Testing

Objective: Evaluate application speed, scalability and stability.

- **Test Case 1:** Application supports (eg: 1000) concurrent users.
Success: System remains responsive.
Failure: Increased response time, unresponsive UI or system crashes.
- **Test Case 2:** Load time (eg: under 2 Seconds).
Success: Page loads within the limit.
Failure: Page loads exceed 2 seconds.

● Security Testing

Objective: Identify vulnerabilities and Data protection

- **Test Case 1:** Passwords are stored securely.
Success: Encrypted passwords in the database.
Failure: Plain text passwords found.
- **Test Case 2:** Application prevents unauthorized access.
Success: Unauthorized access is blocked.
Failure: Unauthorized access is granted.

- **Data Based Testing**

Objective: Verify data base integrity.

- **Test Case 1:** Data is stored correctly upon form submission.
Success: Data saved as expected.
Failure: Data is missing or incorrect.

- **Regression Testing**

Objective: Ensure new updates do not break existing functionality.

- **Test Case 1:** Existing features function post update.
Success: No defectes found.
Failure: Exsiting feture breaks.
- **Test Case2:** Automated scripts validate core functionalities.
Success: All test cases pass.
Failure: Regression failures occur.

- **Compatibility Testing.**

Objective: Ensure application works across multiple environments.

- **Test case 1:** Application runs on different browseres(Chrome,Firefox,Edge).
Success: UI functions consistently.
Failure: UI elements misaligned or break.
- **Test Case 2:** Application works across devices(Mobile,Desktop)
Success: Functionality remains consistent
Failure: Future does not work on specific devices.

- **Usability Testing**

Objective: Ensure the application is user friendly.

- **Test case1:** The interface is easy to navigate.
Success: Users complete tasks effortlessly.
Failure: Users struggle to find key features.

- **Error handling Testing.**

Objective: Validate how the system handles errors.

- **Test Case 1:** User enters invalid data in a application
Success:Error message is displayed.
Failure: System crashes or no error message.
- **Test Case 2:** Network failure during the use of application.
Success: App retries or inform the user.
Failure: Data loss.

4. Test Execution Plan

- **Test Environment Setup**

- Define required devices, OS, browsers and network configuration.

- **Test Data Preparation.**

- Use real data for testing.

- **Test Execution**

- Execute manual and automated test cases.

- **Defect Reporting**

- Track issue in Jira or any other tool.

- **Test Closure**

- Final review and documentation.

5. Conclusion

This test plan ensures comprehensive validation of the application by covering all critical testing aspects. Both manual and automated testing approaches will be employed to achieve robust quality assurance before release.